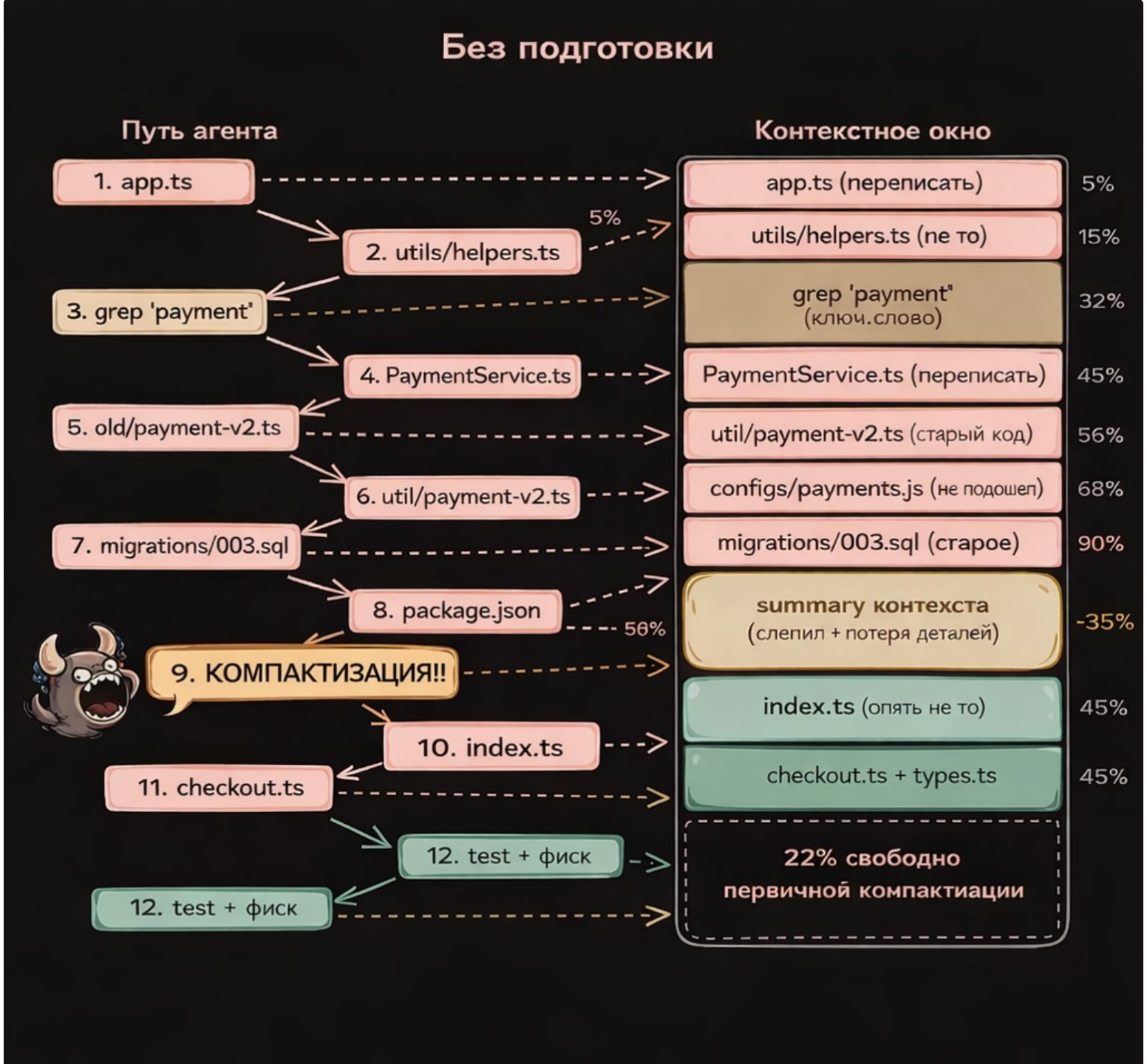
Путь агента Без подготовки



Что в markdown-файлах

- документация
  - роутинг
  - глоссарий
  - архитектура
  - общее описание системы
  - соглашения и вкусы команды
  - гардрейлы и границы

- планы / спецификации
- workflow
- агенты
- команды
- скиллы
- отчеты и исследования
- ADR (Architecture Decision Records)
  - архитектурные ограничения
  - бизнес-правила
  - причины решений

кроме markdown

- OpenAPI
- Swagger
- AsyncAPI
- TypeSpec

Также используются форматы:

- JSON
- YAML
- XML

Хорошая документация не должна повторять код — агенты его и сами неплохо понимают

Она должна:

* давать доменное знание
(всё тайное должно стать markdown)

* отражать намерения и вкусы

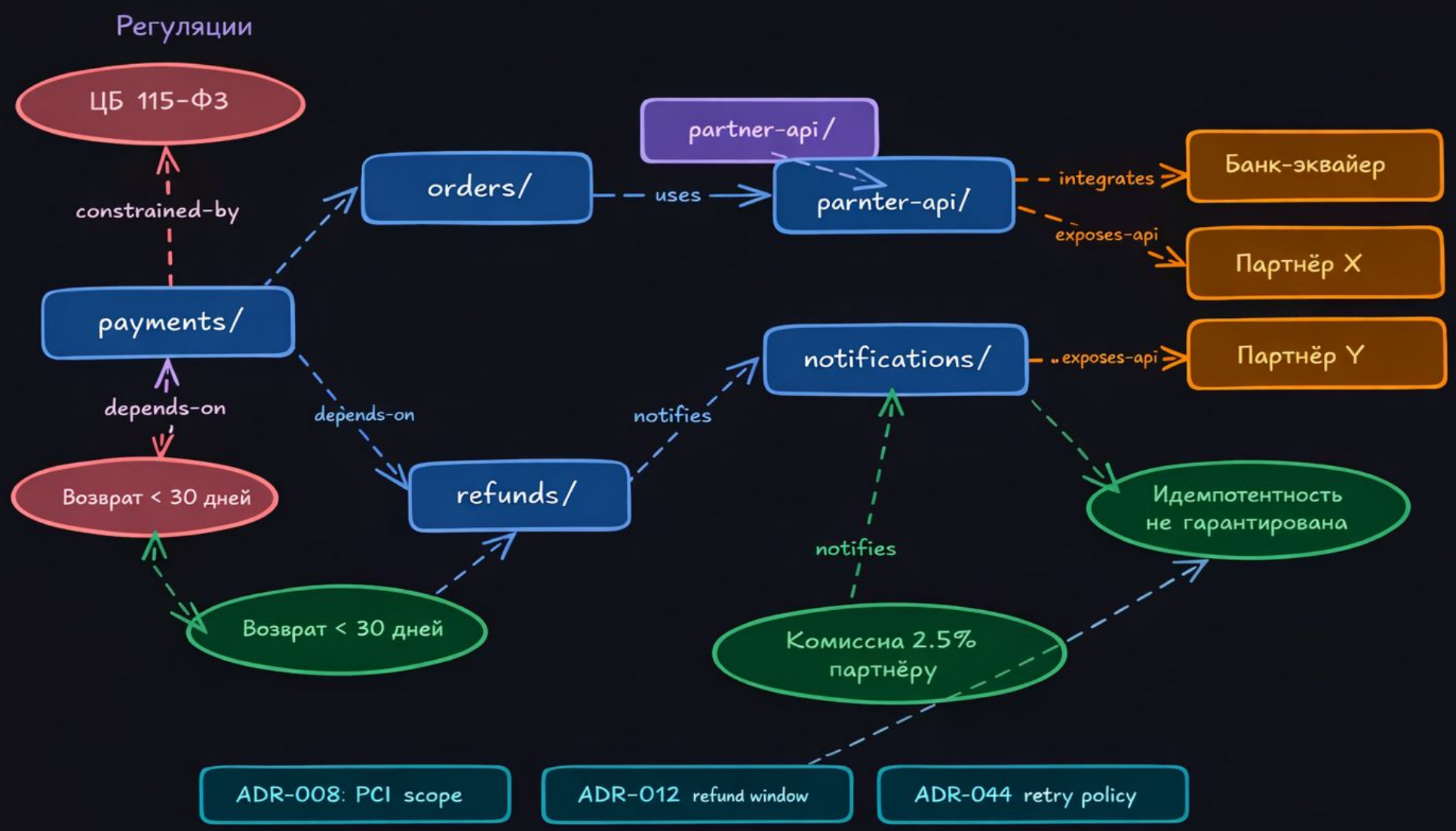
* прояснять неочевидные места

* показывать, как работать с проектом

* быть сжатой проекцией и навигацией

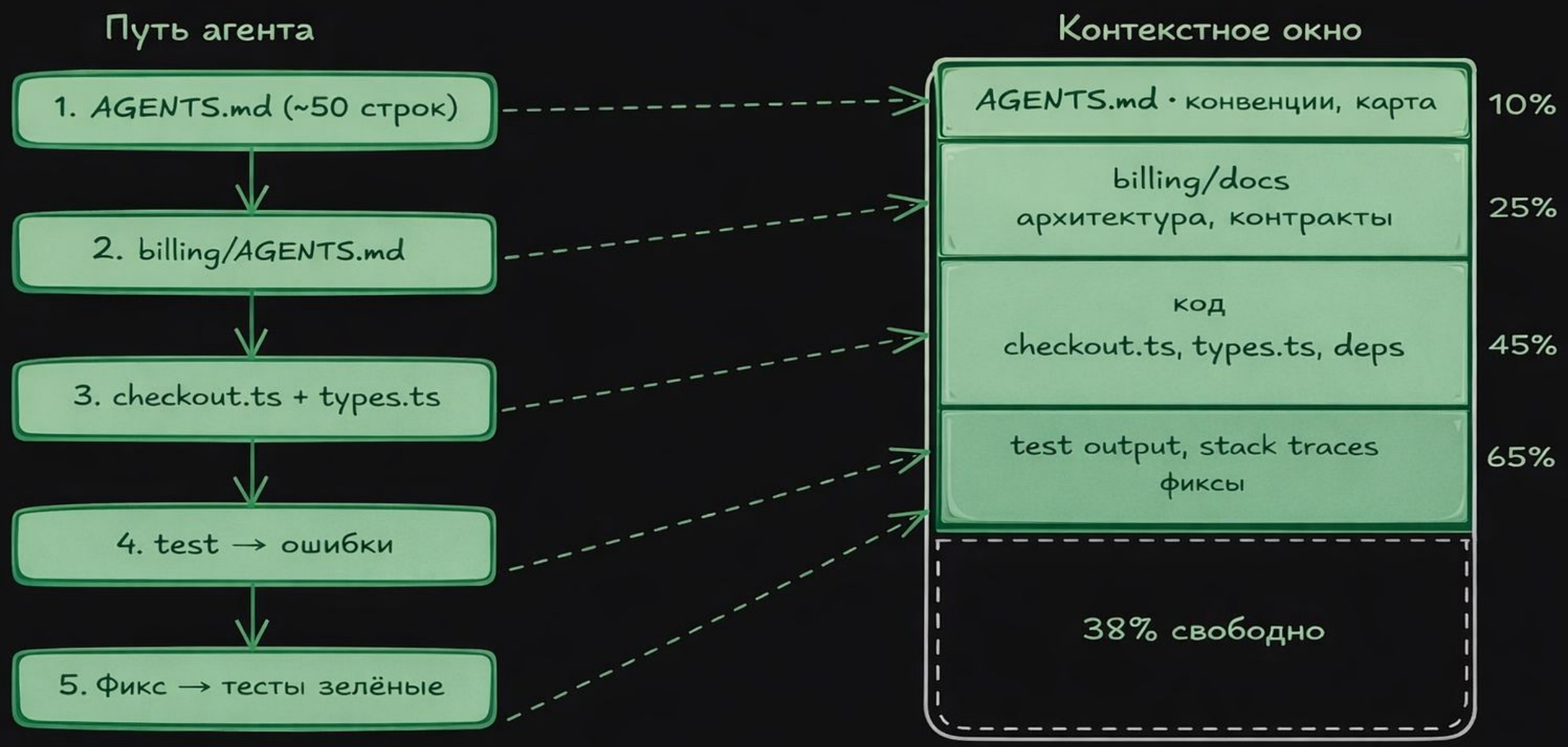
Доменное знание

Доменный граф e-commerce платежей

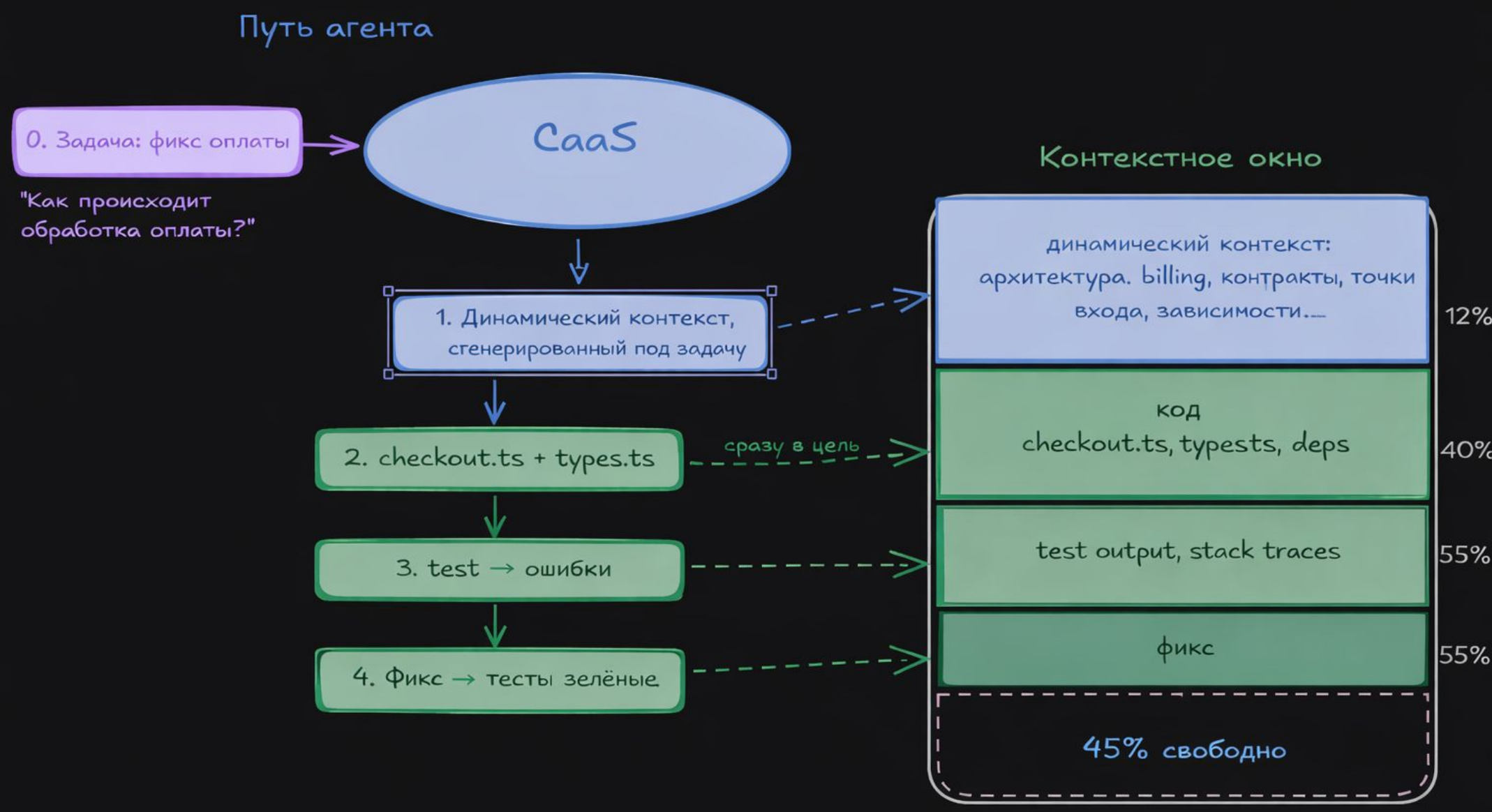


Путь агента с подготовкой

С подготовкой



Путь с Caas


Путь агента
0. Задача: фикс оплаты
"Как происходит обработка оплаты?"
CaaS
1. Динамический контекст, сгенерированный под задачу
2. checkout.ts + types.ts
сразу в цель
3. test → ошибки
4. Фикс → тесты зелёные
Контекстное окно
динамический контекст: архитектура. billing, контракты, точки входа, зависимости...
12%
код checkout.ts, typests, deps
40%
test output, stack traces
55%
фикс
55%
45% свободно

Жизненный цикл документации

Жизненный цикл документации

Как md-файлы деградируют и ломают агента

Как не допускать дрифт

изменил контракт = обнови doc в том же PR + периодический pruning: что устарело — удалить + автоматизация обязательных действий по гигиене

MCP каталог сервисов (корпоративная библиотекой инструментов для всех ИИ-агентов)

Описание

- Создаем единый корпоративный шлюз для подключения внутренних ИТ-систем к ИИ-агентам (Код ассистентам)
- Формируем каталог API-сервисов - стандартный набор инструментов, доступных агентам: описание функций, параметров, ограничений, владельцев
- Стандартизируем механизм вызова внутренних сервисов через MCP
- MCP обеспечивает структурное понимание архитектуры: ИИ-агент знает, какие системы существуют, как они связаны, какие данные и операции доступны, и может безопасно составлять цепочки действий без прямого доступа к системам.

Эффекты

- Агент получает архитектурную модель ИТ сервисов компании, а не “темный лес из API”. Агент становится способен строить корректные цепочки действий, понимать зависимости между системами и автоматически выбирать нужные инструменты
- Снижение интеграционных костов: стандартизованный доступ для всех ИИ-агентов
- Быстрая подключаемость новых систем
- Повышение прозрачности: все взаимодействие ИИ-агентов с ИТ-ландшафтом записано и контролируется
- Повышение надежности работы агентов - меньше ошибок, меньше ручной настройки
- Уменьшение дублирования разработок между командами - один инструмент создается один раз и используется всеми.
  Появление “единого языка взаимодействия”: разработчики, ИИ-агенты и сервисы работают по общему стандарту

Почему это важно

- Сегодня ИИ-агенты не понимают, как устроены ИТ сервисы компания: что можно, что нельзя, какие данные где лежат, какие действия допустимы
- MCP-каталог превращает разрозненные API в структурированное знание об ИТ архитектуре предприятия, доступное агенту: агент понимает что за сервис перед ним, какие у него входы/выходы, какие зависимости, какие риски
- Это ключевое требование для масштабирования ИИ-агентов как код ассистентов: они должны работать осмысленно и безопасно, а не экспериментировать с производственными системами
- Такой подход делает агента инфраструктурно грамотным, что выводит его из уровня усредненного разработчика без знаний контекста устройства ИТ сервисов компании в уровень цифрового инженера, который работает в архитектуре компании осмысленно и управляемо.

MCP библиотека шаблонов и корпоративный стиль кодирования

Описание

- Создаем единый корпоративный набор шаблонов проектов и модулей (Go, Java, Python, TS и др.) с уже встроенными линтерами, форматтерами, CI, политиками безопасности
- Разрабатываем каталог шаблонов, доступных через MCP как инструменты: “создать сервис”, “создать модуль”, “обновить структуру”, “применить стиль кодирования”
- Встраиваем корпоративные стандарты разработки (архитектурные правила, структура каталогов, нейминг, тестовые паттерны, логирование, метрики)
- Делаем эти стандарты исполненными: код ассистент и другие агенты генерируют код сразу по корпоративным нормам, минимизируя ручные ошибки
- Создаем механизм автоматических “рекомендаций по стилю” и рефакторинга через MCP

Эффекты

- Единый стиль и архитектурная дисциплина по всем проектам — вне зависимости от уровня разработчика или команды
- Существенное снижение времени на старт новых сервисов и модулей (быстрый scaffolding “по стандарту”)
- Уменьшение числа дефектов, связанных с неконсистентностью кода, структурой проектов и неподдерживаемыми паттернами
- Автоматическое соблюдение требований ИБ и DevSecOps (структура проекта, CI, зависимости, тесты)
- Повышение производительности разработчиков: меньше рутины, меньше “копипасты”, меньше исправлений после ревью
- Формирование “корпоративного инженерного стандарта”, доступного ИИ-агентам и людям

Почему это важно

- Без единой библиотеки шаблонов ИИ-агенты генерируют код “как попало”, по усредненным интернет-практикам - это приводит к неконтролируемому росту технического долга
- Стандартизация позволяет агентам генерировать сразу пригодный к поддержке код, а не “сырье”, которое нужно переделывать вручную
- Это основа для масштабируемости код агентов: агент должен понимать корпоративные правила, а не только синтаксис языка
- Проект создает фундамент для формирования “корпоративного SDK”, который будет использоваться всеми ИИ-агентами, разработчиками, командами и конвейерами CI
- Повышает качество разработки, скорость и воспроизводимость результаты - ключевые параметры для больших инженерных организаций

MCP сканер уязвимостей

Описание

- Создаем единый корпоративный сервис сканирования зависимостей, библиотек, контейнеров и исходного кода на уязвимости
- Интегрируем сканер в единый оркестратор, доступный через MCP как стандартный инструмент:
  - “проанализировать проект”,
  - “найти уязвимые зависимости”,
  - “предложить обновления”,
  - “сгенерировать безопасный патч”
- Код ассистент и другие ИИ-агенты получают возможность запускать анализ, читать отчеты, автоматически генерировать исправления и обновлять зависимости

Эффекты

- Уменьшение рисков безопасности за счет централизованного и регулярного анализа зависимостей
- Автоматическое исправление уязвимостей с помощью ИИ - сокращение времени на устранение проблем
- Стандартизация процесса анализа во всех командах: единые правила, единые отчеты, единые политики
- Значительное снижение нагрузки на команды разработки и безопасность: рутинная проверка уходит агентам
- Повышение качества проектов: меньше устаревших библиотек, выше соответствие требованиям ИБ
- Возможность автоматического включения проверки в MCP-интеграции, CI/CD, перед релизами или при каждом изменении зависимостей

Почему это важно

- Без централизованного сканера каждая команда живет по своим правилам: разные инструменты, разные частоты проверок, разные уровни риска.
- MCP позволяет ИИ-агентам работать с безопасностью автоматически и предсказуемо, без доступа к системам напрямую.
- Это создает единый стандарт безопасности разработки - критически важно при масштабировании ИИ-агентов на всю компанию
- Проект закрывает ключевой риск: неконтролируемое использование сторонних библиотек и накопление уязвимостей в легаси-коде
- Повышает зрелость организации: от ручного реагирования к превентивной и автоматизированной безопасности